

## ✓ 1. Judul Praktikum

Praktikum Struktur Data - Topik 7: Linked List (Senarai Berantai)

---

## 2. Tujuan Praktikum

- Mempelajari fundamental struktur data memori dinamis yang saling terhubung.
  - Membedakan arsitektur memori antara senarai berantai (Linked List) dengan larik (Array).
  - Menyusun cetak biru (class) untuk objek Node beserta tautannya.
  - Mengimplementasikan operasi sisip, hapus, cari, dan baca pada Singly Linked List.
- 

## 3. Dasar Teori

Linked List adalah bentuk organisasi data linear yang tidak mengandalkan blok memori yang berurutan. Alih-alih menggunakan indeks kaku seperti Array, struktur ini menggunakan simpul (Node). Setiap Node bertugas menyimpan dua hal: nilai data itu sendiri, dan sebuah penunjuk arah (pointer) yang menyimpan alamat memori dari Node berikutnya.

Rantai ini selalu diawali oleh sebuah titik referensi bernama **HEAD** dan diakhiri oleh titik yang menunjuk ke kekosongan (**NULL** atau **None**). Skema dinamis ini sangat menguntungkan komputer karena kita bisa menyelipkan atau mencabut data di tengah jalan tanpa perlu menggeser posisi ribuan data lainnya. Ada beberapa varian yang lazim digunakan, mulai dari *Singly* (satu arah), *Doubly* (dua arah maju mundur), hingga *Circular* (melingkar tanpa ujung).

---

## 4. Langkah Praktikum

Di bawah ini adalah kode untuk merakit kerangka dasar Node dan membangun kelas Singly Linked List yang memuat seluruh fungsi operasional dasar, dilanjutkan dengan simulasi kasus tata kelola daftar pemutaran musik digital.

```
1 print("===== SESI PROSEDUR PRAKTIKUM =====")
2
3 # Membangun Struktur Dasar (Praktikum 1)
4 class Node:
5     def __init__(self, data):
6         self.data = data
7         self.next = None
```

```
8
9 # Membangun Kelas Manajemen Senarai (Praktikum 2 - 6)
10 class LinkedList:
11     def __init__(self):
12         self.head = None
13
14     def tambah_awal(self, data_baru):
15         node_baru = Node(data_baru)
16         node_baru.next = self.head
17         self.head = node_baru
18
19     def tambah_akhir(self, data_baru):
20         node_baru = Node(data_baru)
21         if self.head is None:
22             self.head = node_baru
23             return
24         saat_ini = self.head
25         while saat_ini.next:
26             saat_ini = saat_ini.next
27         saat_ini.next = node_baru
28
29     def hapus_data(self, kunci):
30         saat_ini = self.head
31         sebelumnya = None
32
33         # Jika yang dihapus ada di kepala
34         if saat_ini and saat_ini.data == kunci:
35             self.head = saat_ini.next
36             saat_ini = None
37             return
38
39         # Mencari data di tengah atau akhir
40         while saat_ini and saat_ini.data != kunci:
41             sebelumnya = saat_ini
42             saat_ini = saat_ini.next
43
44         if saat_ini is None:
45             return # Data tidak ketemu
46
47         sebelumnya.next = saat_ini.next
48         saat_ini = None
49
50     def cari_data(self, kunci):
51         saat_ini = self.head
52         posisi = 1
53         while saat_ini:
54             if saat_ini.data == kunci:
55                 return f"Ditemukan pada urutan ke-{posisi}"
56             saat_ini = saat_ini.next
57             posisi += 1
58         return "Data tidak ditemukan"
59
60     def tampilkan_semua(self):
61         saat_ini = self.head
62         hasil = []
63         while saat_ini:
64             hasil.append(str(saat_ini.data))
65             saat_ini = saat_ini.next
66         hasil.append("None")
67         print(" -> ".join(hasil))
68
```

```

69 # Menjalankan Uji Coba Prosedur
70 uji_ll = LinkedList()
71 uji_ll.tambah_akhir(10)
72 uji_ll.tambah_akhir(20)
73 uji_ll.tambah_akhir(30)
74 print("Data awal (Traversal):")
75 uji_ll.tampilkan_semua()
76
77 uji_ll.tambah_awal(5)
78 print("\nSetelah Insert Awal (5):")
79 uji_ll.tampilkan_semua()
80
81 uji_ll.hapus_data(20)
82 print("\nSetelah Delete Tengah (20):")
83 uji_ll.tampilkan_semua()
84
85 print("\nMencari nilai 30:")
86 print(uji_ll.cari_data(30))
87
88
89 print("\n===== SESI STUDI KASUS =====")
90 print("\n--- Kasus 1: Playlist Musik Digital ---")
91 playlist = LinkedList()
92 playlist.tambah_akhir("Lagu A - Pop")
93 playlist.tambah_akhir("Lagu B - Rock")
94 playlist.tambah_akhir("Lagu C - Jazz")
95
96 print("Daftar Putar Saat Ini:")
97 playlist.tampilkan_semua()
98
99 print("\nMenyelipkan lagu baru sebagai pembuka...")
100 playlist.tambah_awal("Lagu Intro - Instrumental")
101 playlist.tampilkan_semua()
102
103 print("\nMenghapus 'Lagu B - Rock' dari daftar...")
104 playlist.hapus_data("Lagu B - Rock")
105 playlist.tampilkan_semua()

```

===== SESI PROSEDUR PRAKTIKUM =====

Data awal (Traversal):

10 -> 20 -> 30 -> None

Setelah Insert Awal (5):

5 -> 10 -> 20 -> 30 -> None

Setelah Delete Tengah (20):

5 -> 10 -> 30 -> None

Mencari nilai 30:

Ditemukan pada urutan ke-3

===== SESI STUDI KASUS =====

--- Kasus 1: Playlist Musik Digital ---

Daftar Putar Saat Ini:

Lagu A - Pop -> Lagu B - Rock -> Lagu C - Jazz -> None

Menyelipkan lagu baru sebagai pembuka...

Lagu Intro - Instrumental -> Lagu A - Pop -> Lagu B - Rock -> Lagu C - Jazz -

```
Menghapus 'Lagu B - Rock' dari daftar...  
Lagu Intro - Instrumental -> Lagu A - Pop -> Lagu C - Jazz -> None
```

---

## 5. Analisis

Pada rancangan kode di atas, objek Node bertindak selayaknya gerbong kereta, sedangkan atribut `next` adalah pengait antargerbong tersebut. Berbeda dengan Array yang perlu memindahkan isi gerbong jika ada perubahan, pada senarai berantai kita hanya perlu membongkar pasang pengaitnya saja. Operasi sisip di bagian kepala sangat efisien karena beroperasi seketika. Namun, untuk menambah data di bagian ekor, sistem terpaksa melakukan penelusuran dari awal hingga bertemu ujung `None`.

---

## 6. Kesimpulan

Senarai berantai memberikan solusi brilian untuk sistem yang membutuhkan ukuran memori elastis. Meskipun ia harus mengorbankan sedikit kecepatan pada proses pencarian acak, keunggulannya dalam melakukan modifikasi data di tengah rantai menjadikannya arsitektur wajib untuk membangun aplikasi dunia nyata seperti riwayat peramban web hingga fungsi pemulihan (*Undo*).